K.R. MANGALAM UNIVERSITY

School of Engineering & Technology
Department of Computer Science & Engineering

ENSP465 — New Age Programming Languages Lab
Phase 1 Project Report

High-Performance Concurrent Web Crawler and Local Inverted Index Desktop Search Engine in Go

Students:	• Naman Katewa (Roll No: 2401830002) — B.Sc. (Hons.) Cyber Security [Team Lead] • K Sai Vignesh Yadav (Roll No: 2401720006) — B.Sc. (Hons.) Computer Science • Hrishit Kaushal (Roll No: 2501722016) — B.Sc. (Hons.) Computer Science
Semester & Academic Year:	Semester 5th, Academic Year 2026-2027
Course Faculty:	Dr. Hemraj Kumawat School of Engineering & Technology, KRMU

Section 2: Problem Statement & Motivation

2.1 Problem Description

Modern search systems depend almost entirely on centralized cloud infrastructure. Whether using commercial search engines like Google or hosted enterprise search APIs, users must transmit search queries over external networks. In air-gapped security research labs, confidential corporate intranets, and remote industrial environments, continuous internet access is either physically unavailable or forbidden by compliance policy. When working with local technical manuals, offline documentation, and web corpora, engineers lack an autonomous tool that can crawl, index, and search content locally with mathematical relevance ranking.

Transmitting search queries across the public internet introduces telemetry and data leakage risks. Third-party cloud engines log search queries, build user tracking profiles, and expose proprietary technical interests. Existing desktop search tools fail to solve this problem. Desktop file utilities like Everything index file names and metadata, but they do not crawl web pages, strip HTML boilerplate, or rank results based on body text relevance. Conversely, enterprise engines like Elasticsearch and Apache Solr run on the Java Virtual Machine. They require gigabytes of baseline memory, continuous daemon processes, and complex cluster configurations, making them impractical for a lightweight desktop utility.

A clear gap exists between simple file-name finders and heavy enterprise search servers. Technical users require a self-contained desktop search engine that runs locally on consumer workstations. It must crawl local or remote sites using concurrent workers, cleanly strip HTML boilerplate, build a local inverted index, and rank search queries instantly using algorithmic models, all without external cloud dependencies or background daemon services.

2.2 Real-World Context and Domain

This project operates in systems programming, concurrent network I/O, and information retrieval. It serves four operational contexts:

1. Air-Gapped and Secure Networks: Security testing labs, defense systems, and industrial environments where internet access is prohibited.

2. Offline Technical Documentation: Software engineers needing rapid full-text search across downloaded API references, RFCs, and offline technical sites.

3. Intranet Document Retrieval: Research teams and university labs indexing local departmental web portals without deploying server clusters.

4. Systems Concurrency Study: A practical demonstration of lightweight threading, channel-based synchronization, and memory-safe probabilistic structures.

2.3 Motivation

Modern search applications have become bloated with cloud dependencies, neural network runtimes, and telemetry agents. For direct technical keyword lookup, classic information retrieval algorithms like Term Frequency-Inverse Document Frequency (TF-IDF) remain fast, transparent, and accurate. Building this system in Go demonstrates how a compiled systems language handles thousands of concurrent web requests, thread-safe database batching, and instant state management without external runtimes.

2.4 Existing Tools and Their Limitations

Table 1 outlines how existing search solutions compare against the proposed Go desktop search engine.

Feature / Criterion	Cloud Search (Google)	Enterprise (Elasticsearch)	Desktop Tool (Everything)	Proposed Go Engine
Deployment Model	Remote Cloud / SaaS	JVM Cluster Daemon	Local Desktop Utility	Local Desktop Executable
Network Need	Continuous Internet	Local/Remote Network	None (Local Disk Only)	Optional (Local Crawl / Offline)
RAM Footprint	Negligible on Client	High (2 GB to 8 GB+)	Minimal (< 100 MB)	Low (< 150 MB RAM)
Content Indexing	Full Web Indexing	Full Text & Schema	File Names Only	Clean Full-Text Inverted Index

Table 1: Comparative analysis of existing search tools versus the proposed Go engine.

Section 3: Proposed Solution & Language Justification

3.1 Project Overview

The proposed system is an autonomous desktop search engine packaged as a single executable using Wails v2 and Next.js. It features a concurrent Go web crawler that fetches web pages simultaneously using goroutines, an HTML parser that strips boilerplate navigation markup, an

in-memory Bloom filter to track visited links, and a dedicated database writer that commits words to a local SQLite inverted index. Search queries calculate TF-IDF relevance scores on the fly, delivering ranked results in under 50ms.

3.2 Language Selection: Go (Golang)

From the ENSP465 curriculum (comprising Go, F#, Clojure, and Kotlin), we selected Go. Go was developed specifically for concurrent systems engineering, network servers, and high-performance system utilities.

3.3 Language Justification Against Curriculum Alternatives

Go provides four specific technical advantages over Kotlin, Clojure, and F# for building this search engine:

1. Goroutines and M:N Cooperative Scheduling

Web crawling is an I/O-bound workload requiring thousands of concurrent network connections. In Kotlin on the JVM, threads map 1:1 to operating system kernel threads, each allocating roughly 1 MB of stack memory. Spawning 2,000 threads consumes 2 GB of memory in stack frames alone, inducing severe context-switching overhead. Go employs goroutines managed by an internal M:N runtime scheduler. Goroutines start with a 2 KB stack that expands dynamically. Spawning 2,000 crawler workers in Go consumes under 25 MB of memory, enabling high crawl concurrency on standard desktop hardware.

2. CSP Concurrency via Buffered Channels

SQLite enforces single-writer serialization: multiple threads attempting concurrent writes trigger 'database is locked' errors. In shared-memory languages, coordinating concurrent writes requires explicit mutexes and condition variables, risking deadlocks. Go implements Communicating Sequential Processes (CSP). The crawler swarm sends parsed documents over a buffered Go channel (chan ParsedDocument). A single dedicated writer goroutine reads from this channel and commits batches of 50 documents in single transactions. This completely eliminates lock contention without complex mutex synchronization.

3. Low-Latency Garbage Collection and Native Binary Compilation

JVM languages like Kotlin and Clojure require a heavy runtime, suffer from cold-start latency, and experience stop-the-world garbage collection pauses during heavy text tokenization. Clojure's immutable structures also generate significant heap churn. Go compiles directly to

native machine code with zero virtual machine startup overhead. Go gives direct control over memory layout through value types and slices, avoiding object allocation overhead and keeping garbage collection pauses below 1 millisecond.

4. Fullstack Desktop Type Safety via Wails v2

Desktop development in F# or Clojure typically requires heavy Electron wrappers that consume hundreds of megabytes of RAM. Wails v2 couples Go with native OS webviews (Microsoft Edge WebView2 on Windows). Wails parses Go structs at compile time and generates matching TypeScript interfaces, establishing compile-time type safety across the Go backend and Next.js frontend.

3.4 Key Libraries and Frameworks

• **Wails v2 (github.com/wailsapp/wails/v2):** Packages the Go backend and Next.js UI into a single desktop executable with bidirectional IPC bindings.

• **net/http (Standard Library):** Provides production-grade HTTP client connection pooling and timeout management.

• **golang.org/x/net/html:** Standard HTML5 tokenizer used to strip script tags and isolate textual body content.

• **modernc.org/sqlite:** A pure Go SQLite engine requiring no external C compiler (CGo-free), enabling simple cross-platform builds.

Section 4: Objectives & Scope

4.1 Specific, Measurable Objectives

The project commits to five concrete engineering targets:

1. Crawl Throughput: Achieve a sustained crawling rate of at least 50 pages per second on local network targets using concurrent goroutines.

2. Memory-Safe URL Tracking: Restrict visited URL memory consumption to under 5 MB for up to 1,000,000 links using an in-memory Bloom filter with false-positive rate below 0.1%.

3. Sub-50ms Search Latency: Execute query tokenization, term lookup, dynamic IDF calculation, and TF-IDF ranking in under 50 milliseconds across 100,000 indexed records.

4. Automated Scraping Politeness: Enforce 100% compliance with target domain robots.txt disallow directives and per-domain crawl delays.

5. Standalone Desktop Binary: Produce a single standalone executable under 30 MB that operates without external runtime installations.

4.2 In-Scope Features (Phase II and Phase III)

• **Phase II (Core Backend):** Concurrent crawler swarm, Bloom filter visited set, robots.txt evaluator, HTML readability noise filter, buffered Go channel pipeline, and dedicated SQLite batch writer.

• **Phase III (Search, UI, and Testing):** Dynamic query-time TF-IDF ranking engine, Next.js dashboard with live metric event streaming, instant search bar, unit and integration test suite, and performance benchmarks.

4.3 Out-of-Scope Items

• **Cloud Synchronization & Clustering:** No multi-node distributed database replication or remote cloud backups.

• **Neural NLP and Large Language Models:** No deep neural networks or generative AI APIs. Ranking relies strictly on mathematical TF-IDF.

• **Multi-User Authentication:** No user login accounts or role-based access control. The app runs as a single-user workstation tool.

• **Headless JavaScript Execution:** The crawler processes raw static HTML byte streams without embedding heavy browser engines like Chromium.

Section 5: System Architecture & Design

5.1 High-Level Architecture Overview

The system follows a decoupled three-tier architecture: the Presentation Layer (Next.js desktop UI), the IPC Runtime Bridge (Wails v2), and the Core Backend Systems Engine (Go runtime). Figure 1 illustrates the high-level block diagram and component connections.

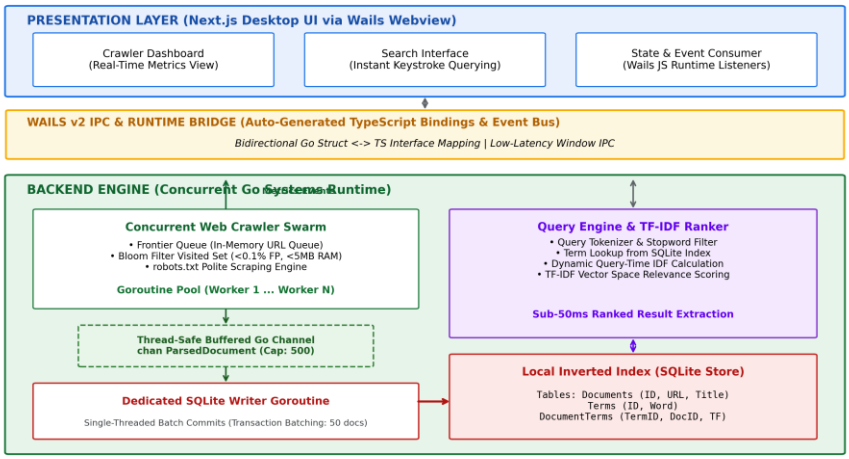


Figure 1: High-Level System Architecture and Concurrency Model.

5.2 Major Component Subsystems

The system is organized into eight primary modules:

1. Frontier Queue: Thread-safe in-memory FIFO queue that normalizes discovered URLs and schedules them for worker fetching.

2. Bloom Filter (Visited Set): Probabilistic bitset tracking visited links in O(k) time, preventing link cycles while consuming under 5 MB of RAM.

3. Polite Scraping Engine: Caches domain robots.txt files, verifies crawler permissions, and applies rate-limiting delays.

4. Concurrent Worker Swarm: Lightweight goroutines that pop URLs from the frontier and stream HTTP response bodies over pooled TCP connections.

5. HTML Parser & Noise Removal: Strips scripts, styling, navigation bars, and footers from HTML streams, isolating readable body text and child links.

6. Buffered Channel Pipeline: A thread-safe channel (chan ParsedDocument, capacity 500) that buffers documents and isolates network workers from disk I/O.

7. Dedicated SQLite Batch Writer: A single goroutine that drains the channel and writes batches of 50 documents per database transaction, preventing locks.

8. Query Engine & TF-IDF Ranker: Tokenizes query strings, retrieves matching term counts from SQLite, computes dynamic IDF scores, and returns sorted results.

5.3 System Lifecycle and Data Flow (DFD Level 1)

Figure 2 illustrates the data flow diagram (DFD Level 1) for both the web crawling pipeline and the search retrieval flow.

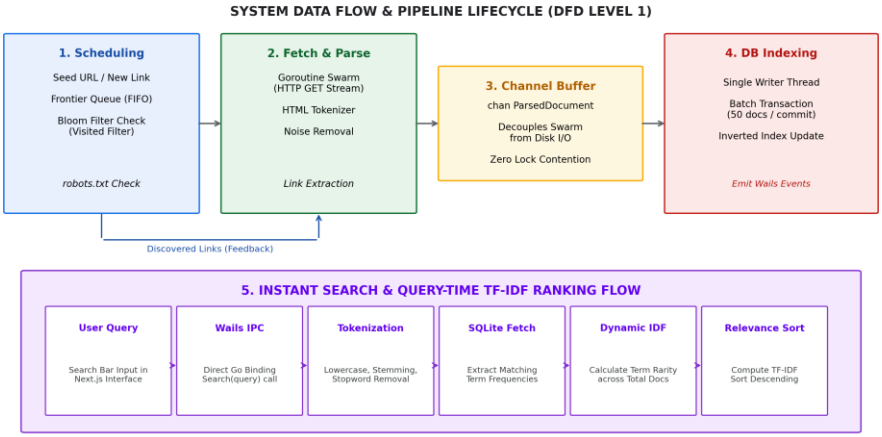


Figure 2: System Data Flow and Pipeline Lifecycle (DFD Level 1).

5.4 Input-Process-Output (IPO) Analysis

Table 2 outlines the operational input, internal processing, and generated output across the three core operational phases.

Pipeline Stage	Input Data	Internal Processing	Output Produced
Web Crawling	Seed URL, Max Pages limit, Domain scope	Frontier queue dispatch, Bloom filter check, robots.txt verification, HTTP GET streaming	Raw HTML response streams, child URLs added to frontier
Content Indexing	Raw HTML stream, Document URL	HTML tokenization, script/boilerplate stripping, term frequency calculation, SQLite batch commit	Updated SQLite Inverted Index (Documents, Terms, DocumentTerms)
Interactive Search	Search keywords from user interface	Query tokenization, SQLite term lookup, dynamic IDF calculation, TF-IDF dot product, sort	Ranked list of matching URLs, page titles, and relevance scores

Table 2: Input-Process-Output (IPO) specification across system lifecycle stages.

5.5 Concurrency Primitives and Mathematical Formulations

1. Goroutines and context.Context Cancellation: The master crawler coordinates workers via a sync.WaitGroup and a context.Context. When the user terminates a crawl, context cancellation propagates across all active worker goroutines, closing HTTP sockets gracefully.

2. Bloom Filter Sizing Mathematics: To track n expected URLs with a false-positive probability p, the required bit array size m and optimal hash function count k are derived mathematically via Equations 1 and 2:

$$m = -\frac{n \cdot ln(p)}{(ln\,2)^2} \quad (1)$$

$$k = \frac{m}{n} \cdot ln(2) \quad (2)$$

For n = 1,000,000 URLs and a strict false-positive threshold p = 0.001 (0.1%), evaluating Equation 1 yields m = 14,377,588 bits (~1.71 MB). Evaluating Equation 2 gives k = 10 hash functions. The Bloom filter thus guarantees sub-2 MB memory consumption across one million URLs.

3. Query-Time TF-IDF Mathematical Ranking: Rather than pre-computing static weights, relevance ranking is calculated dynamically at query time. Term Frequency (TF) for term t in document d is normalized by document length in Equation 3:

$$TF(t,d) = \frac{f_{t,d}}{\sum f_{t\prime,d}} \quad (3)$$

Smoothed Inverse Document Frequency (IDF) accounts for term rarity across the total document corpus D in Equation 4:

$$IDF(t,D) = ln\left(\frac{1 + |D|}{1 + |\{d \in D : t \in d\}|}\right) + 1 \quad (4)$$

The composite relevance score for query q against candidate document d is the inner product defined in Equation 5:

$$Score(q,d) = \sum [\, TF(t,d) \cdot IDF(t,D) \,] \quad (5)$$

5.6 SQLite Inverted Index Schema Design

The database schema uses three normalized tables with B-Tree indexes:

• **Documents Table:** id INTEGER PRIMARY KEY AUTOINCREMENT, url TEXT UNIQUE NOT NULL, title TEXT, crawled_at TIMESTAMP.

• **Terms Table:** id INTEGER PRIMARY KEY AUTOINCREMENT, word TEXT UNIQUE NOT NULL (Indexed with UNIQUE B-Tree on word).

• **DocumentTerms Table:** term_id INTEGER, document_id INTEGER, term_frequency INTEGER, PRIMARY KEY (term_id, document_id), FOREIGN KEY (term_id) REFERENCES terms(id), FOREIGN KEY (document_id) REFERENCES documents(id).

Section 6: Work Plan & Timeline

6.1 Milestone Schedule

The project maps across six milestones from Phase I submission to final Phase III examination, detailed in Table 3.

Milestone	Deliverable Description	Phase Mapping	Target Date
M1	Problem definition, system architecture design, Phase 1 documentation submission	Phase I (Design & Feasibility)	October 19, 2026
M2	Wails v2 desktop shell setup, Next.js frontend scaffolding, Go struct TypeScript bindings	Phase II (Core Implementation)	November 5, 2026
M3	Goroutine crawler swarm, in-memory Bloom filter, robots.txt polite scraping engine	Phase II (Core Implementation)	November 20, 2026
M4	HTML readability tokenizer, buffered Go channel pipeline, SQLite batch indexer	Phase II (Core Implementation)	December 5, 2026
M5	Dynamic query-time TF-IDF ranking engine, instant search bar, real-time Wails metrics	Phase III (Search & UI Polish)	December 20, 2026
M6	Unit/integration testing, performance benchmarking, final documentation, viva presentation	Phase III (Evaluation & Viva)	January 15, 2027

Table 3: Work plan milestone schedule mapping Phase I through Phase III.

6.2 Phase Deliverables Breakdown

• Phase I Deliverables (Completed): Formal project proposal, architectural block diagram, DFD Level 1, language justification, and complete Phase 1 documentation report.

• Phase II Deliverables (Target: December 2026): Functional concurrent crawler in Go, in-memory Bloom filter, HTML noise stripper, buffered channel pipeline, and atomic SQLite batch writer.

• Phase III Deliverables (Target: January 2027): Dynamic TF-IDF ranking engine, real-time Wails UI dashboard with streaming metrics, unit and integration test suite, performance benchmark report, and final viva presentation.

6.3 Task Distribution Across Team Members

Table 4 outlines the module ownership across team members:

Team Member	Program / Roll No	Primary Architectural Responsibilities
Naman Katewa (Project Lead)	B.Sc. (H) Cyber Security Roll No: 2401830002	Overall systems architecture, concurrent crawler swarm, in-memory Bloom filter visited set, robots.txt polite scraping compliance, and Wails IPC runtime bridge.
K Sai Vignesh Yadav	B.Sc. (H) Computer Science Roll No: 2401720006	SQLite inverted index schema design, single-threaded batch writer goroutine, buffered Go channel pipeline, and HTML readability noise removal module.
Hrishit Kaushal	B.Sc. (H) Computer Science Roll No: 2501722016	Query tokenization, dynamic query-time TF-IDF relevance ranking algorithms, Next.js frontend UI dashboard, real-time Wails event integration, and test suite execution.

Table 4: Team task allocation and module ownership matrix.

Section 7: References

All references adhere to the IEEE citation standard:

[1] A. Donovan and B. Kernighan, The Go Programming Language, 1st ed. Boston, MA, USA: Addison-Wesley Professional, 2015.

[2] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," Communications of the ACM, vol. 13, no. 7, pp. 422-426, Jul. 1970, doi: 10.1145/362686.362692.

[3] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513-523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[4] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, UK: Cambridge University Press, 2008.

[5] Wails Development Team, "Wails: Build desktop applications using Go and Web Technologies," Wails Documentation, 2026. [Online]. Available: https://wails.io/docs/introduction. [Accessed: October 2026].

[6] D. Vyukov, "Go Preemptive Scheduler Design Document," Go Programming Language Wiki, 2019. [Online]. Available: https://go.googlesource.com/proposal/+/master/design/24543-non-cooperative-preemption.md. [Accessed: October 2026].

[7] D. R. Hipp, "SQLite Architecture and Concurrency Locking Mechanisms," SQLite Consortium, 2026. [Online]. Available: https://www.sqlite.org/lockingv3.html. [Accessed: October 2026].